

NarrowAisleBot

The ROS 2 software on the robot: every node, what it does, and why it is there

With the current mapping (slam_toolbox), Nav2 and MPPI controller status

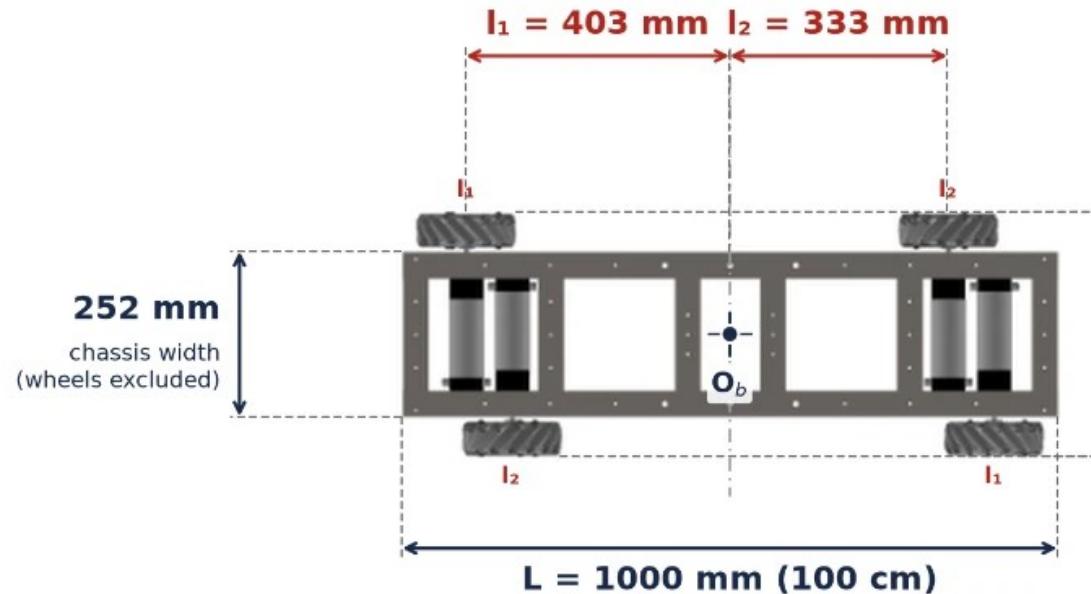
Aritra Das

IIT Bombay | October 2026

Robot measurements as of 1 Oct 2026



What the robot is, and what the software has to do



Wheel layout from the CAD model. Outer wheels (FR, RL) sit 403 mm from the centre, inner wheels (FL, RR) 333 mm. The chassis is 252 mm wide without wheels, about 360 mm across them.

Why the wheelbase is asymmetric

Staggering the wheel pairs lets the chassis be narrow enough for aisles a conventional mecanum platform cannot enter.

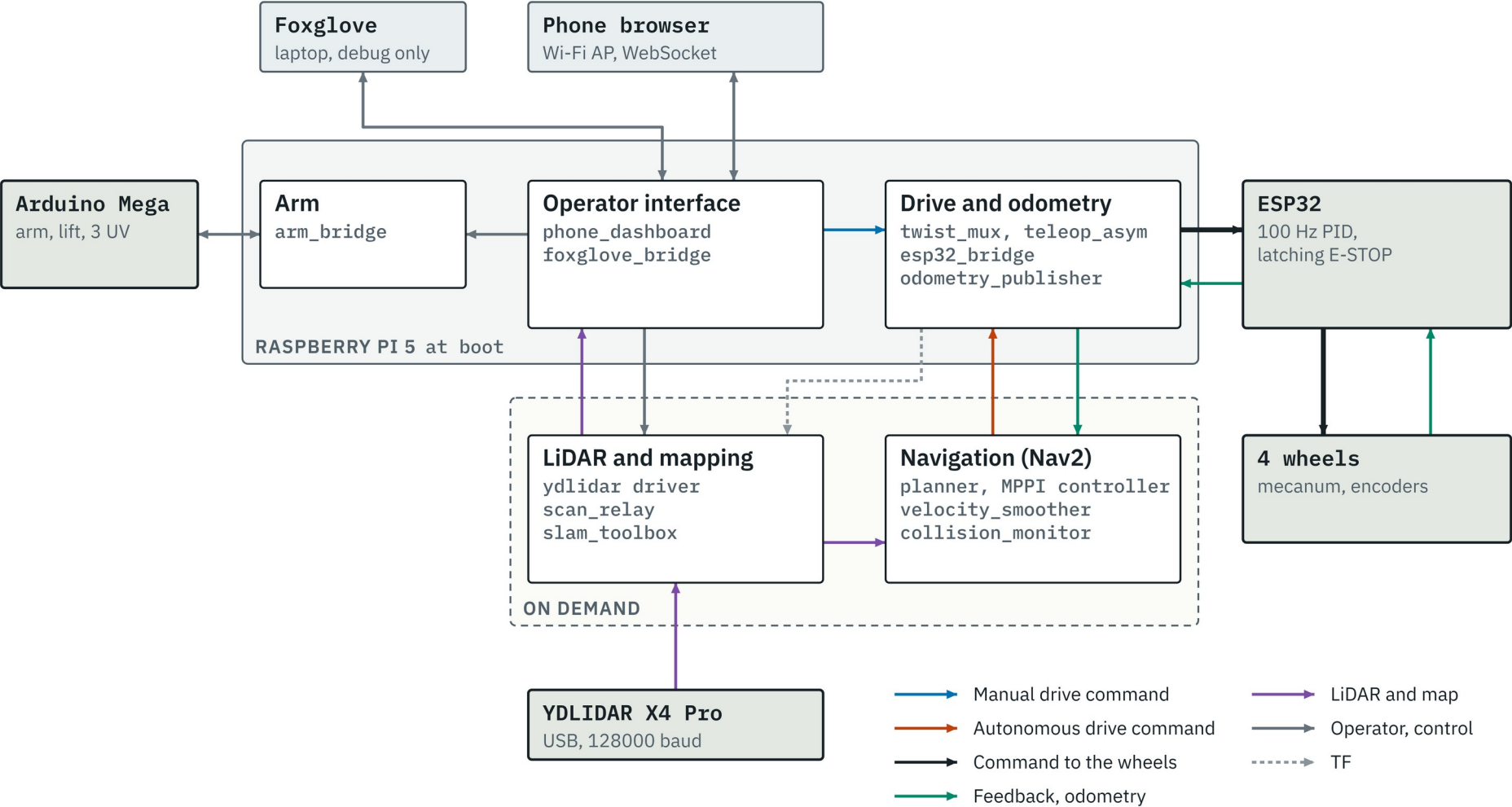
What the software must do

Drive in any direction, sideways and in reverse included, from a phone and on its own in unknown space, while carrying a UV lamp arm.

How the work is split

A 100 Hz wheel-speed loop on an ESP32, the arm on an Arduino Mega, and everything else as ROS 2 nodes on a Raspberry Pi 5.

The whole system on one page



Three computers, each with one job

Raspberry Pi 5

Ubuntu 24.04, ROS 2 Jazzy

11

processes at boot, more on demand

- Operator interface, kinematics, odometry, mapping and planning
- Runs its own Wi-Fi access point for the phone
- Talks to both microcontrollers over USB serial

Why: Gives the libraries (Nav2, slam_toolbox) but cannot promise a 10 ms loop.

ESP32-WROOM-32

Drive controller, firmware v3.0

100 Hz

wheel-speed PID on four motors

- Counts encoders in hardware (PCNT), 186,264 counts per front wheel turn
- Latching E-STOP; overspeed, runaway and stall trips
- 921,600 baud link to the Pi

Why: Timing and safety that do not depend on Linux being responsive.

Arduino Mega 2560

Arm and UV controller, firmware v8

3

stepper axes: two arms and a lift

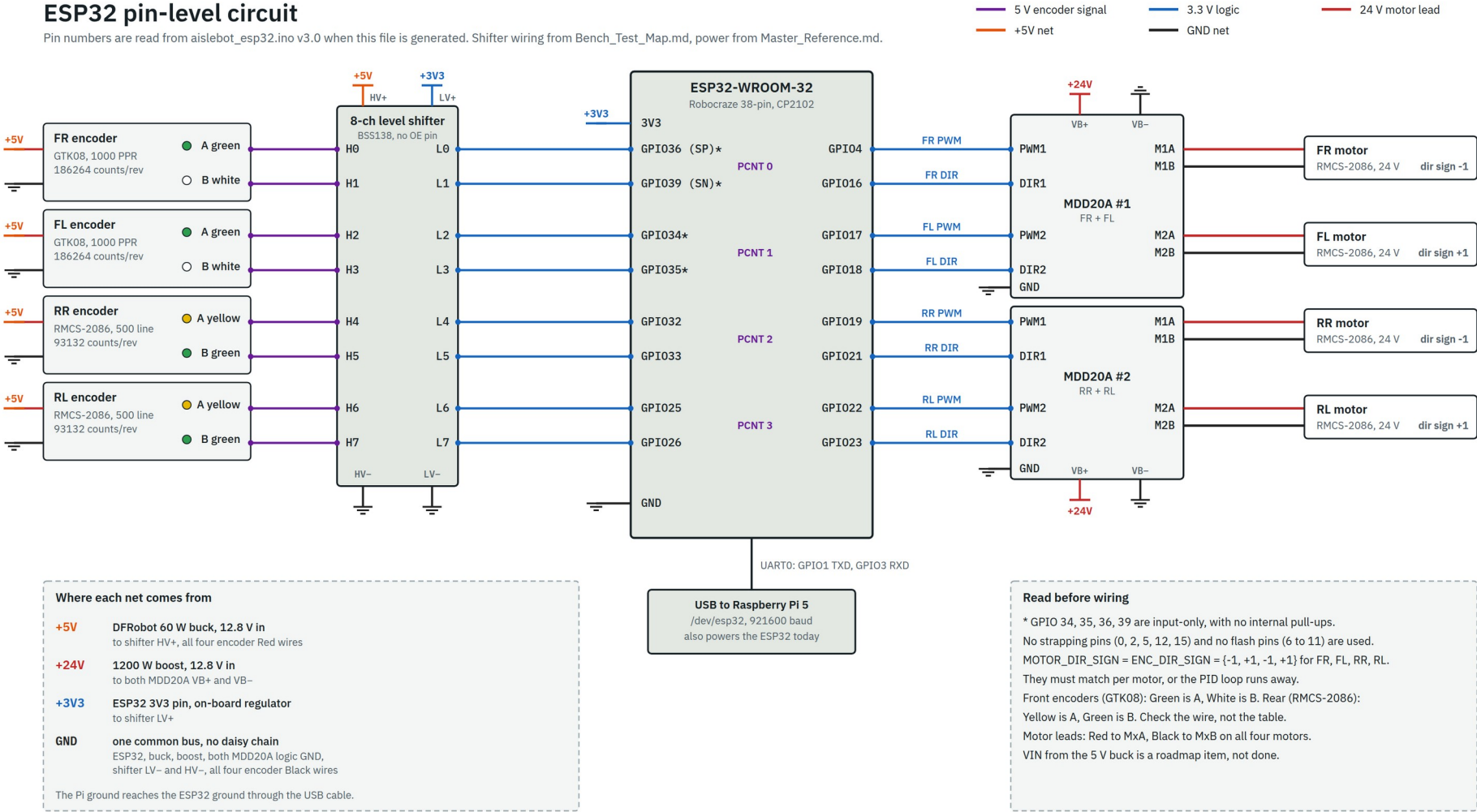
- Three UV tubes on relays, switched on in stages
- 500 ms watchdog on arm motion
- 115,200 baud link to the Pi

Why: Step pulses and relay switching stay off the Pi.

How the ESP32 is wired

ESP32 pin-level circuit

Pin numbers are read from aislebot_esp32.ino v3.0 when this file is generated. Shifter wiring from Bench_Test_Map.md, power from Master_Reference.md.



Not checked against the physical robot. Drawn from the firmware and the repo wiring docs.

Power rails and grounding

Rail	Source	Feeds
12.8 V	LiFePO4 battery, 30 Ah, through an SSR-50DD solid-state relay	Boost and buck converters
24 V	1200 W boost converter	Both Cytron MDD20A motor drivers
5 V	DFRobot 60 W buck converter	Four encoders, level shifter high side
3.3 V	ESP32 on-board regulator	Level shifter low side
USB 5 V	Raspberry Pi 5	ESP32, power and data together

One ground bus

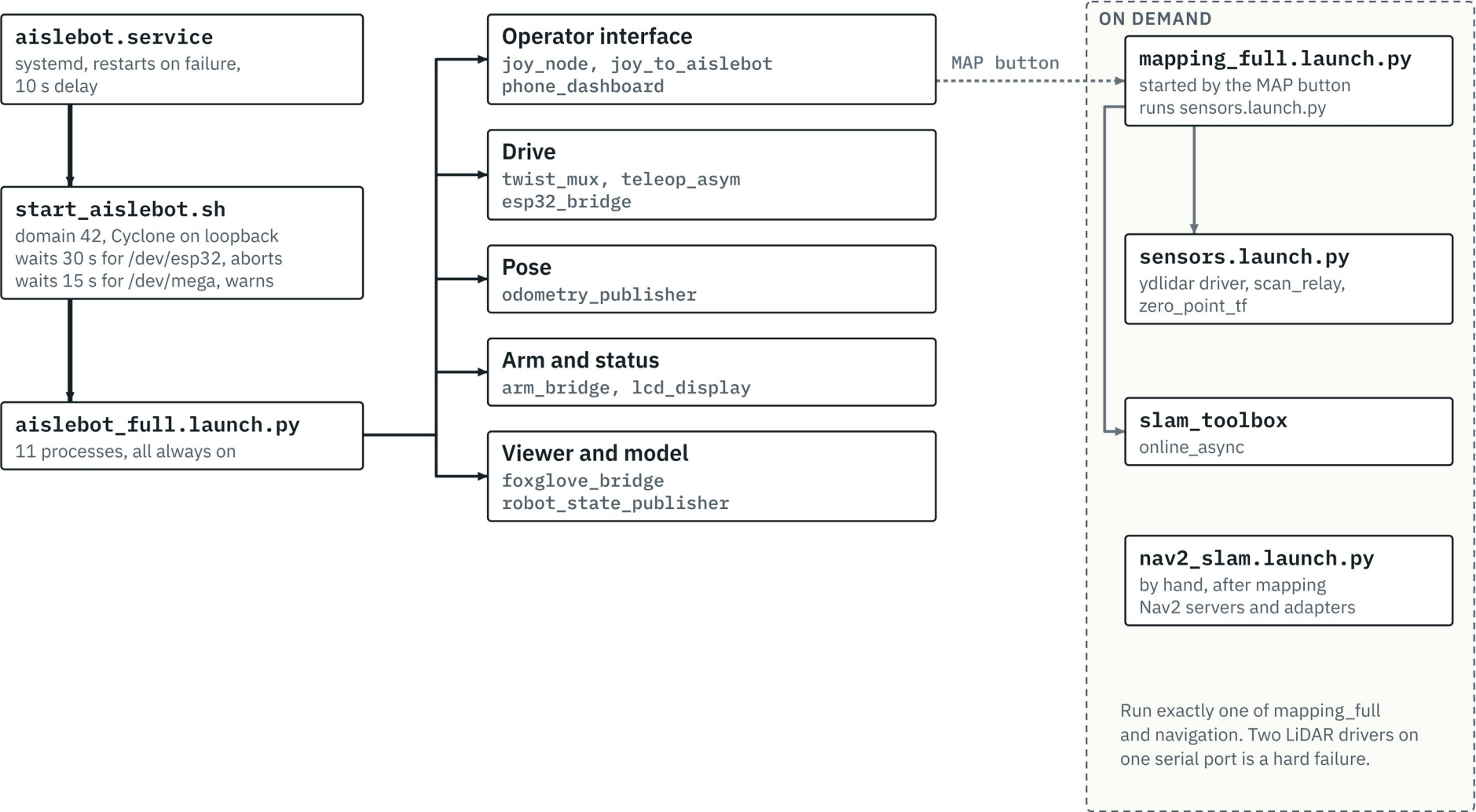
ESP32, buck, boost, both driver logic grounds, the shifter and all four encoder black wires. The Pi's ground arrives through the USB cable.

Not done yet

No hardware E-stop. Every stop path is software today. A latching normally-closed button in series with the SSR trigger is planned.

The ESP32 is still powered from the Pi's USB. Feeding its VIN from the 5 V buck and cutting VBUS is planned, to keep motor ground noise out of the encoder counts.

What starts, and when



Running at boot: operator side and drive chain

Node	What it does	Why it is there
phone_dashboard written for this robot	Web page served to the phone over the robot's own Wi-Fi: joystick, E-STOP, arm and UV buttons, live map and scan, tap-to-goal, start and stop mapping, CSV logs.	The phone is the operator console. No laptop or gamepad is needed on the floor.
joy_node, joy_to_aislebot	Read a USB gamepad and turn sticks and buttons into drive and arm commands.	Backup control path. No gamepad is fitted at present.
twist_mux	Chooses between manual and Nav2 commands. Manual has priority 100, Nav2 10; each source is dropped after 0.5 s of silence.	Manual control must always beat the planner, with no fighting over the wheels.
teleop_asym written for this robot	Inverse kinematics for the asymmetric base: body velocity (vx, vy, wz) to four wheel speeds.	The wheelbase is not a rectangle, so the stock mecanum equations would be wrong.
esp32_bridge written for this robot	Serial link to the ESP32: sends wheel-speed setpoints, reads 20 Hz telemetry, forwards E-STOP. Sends zero speeds after 0.5 s without input.	Keeps the real-time loop on the microcontroller and everything else in ROS.

Running at boot: pose, arm and support

Node	What it does	Why it is there
odometry_publisher written for this robot	Integrates measured wheel speeds into position and heading. Publishes /wheel_odom and the odom to base_link transform.	The only pose source on the robot. There is no IMU, and SLAM scan matching is off.
arm_bridge written for this robot	Serial link to the Mega: arm and lift velocity, homing, the UV tube sequence. Its own 300 ms watchdog.	Stepper timing and relay switching stay off the Pi.
lcd_display written for this robot	Shows the Pi's IP address and network mode on the 16x2 LCD every 2 s.	The robot runs headless; this is how you find it on the network.
robot_state_publisher	Publishes the fixed transforms in the URDF, including where the LiDAR sits on the body.	SLAM and Nav2 need the sensor's position relative to the robot centre.
foxglove_bridge	Streams every topic to Foxglove Studio on a laptop over a WebSocket.	Debug only: the one place to see costmaps, the plan and the collision monitor live.

Started with mapping: LiDAR and slam_toolbox

Node	What it does	Why it is there
ydlidar driver	Talks to the YDLIDAR X4 Pro over USB and publishes /scan, measured at about 11.4 Hz.	Vendor driver, used as supplied.
scan_relay written for this robot	Undoes the sensor's mirrored bearing, blanks the 90 degree arc hidden by the rear mast, and republishes the scan as RELIABLE.	Without it the map comes out mirrored, the mast appears as an obstacle, and SLAM and the costmaps (which subscribe RELIABLE) receive nothing.
slam_toolbox	Builds the occupancy-grid map from scans and odometry; publishes /map and the map to odom correction.	The robot works in unknown space and needs a map to plan on.
zero_point_tf	Static marker at the map origin.	A repeatable start position for every test.

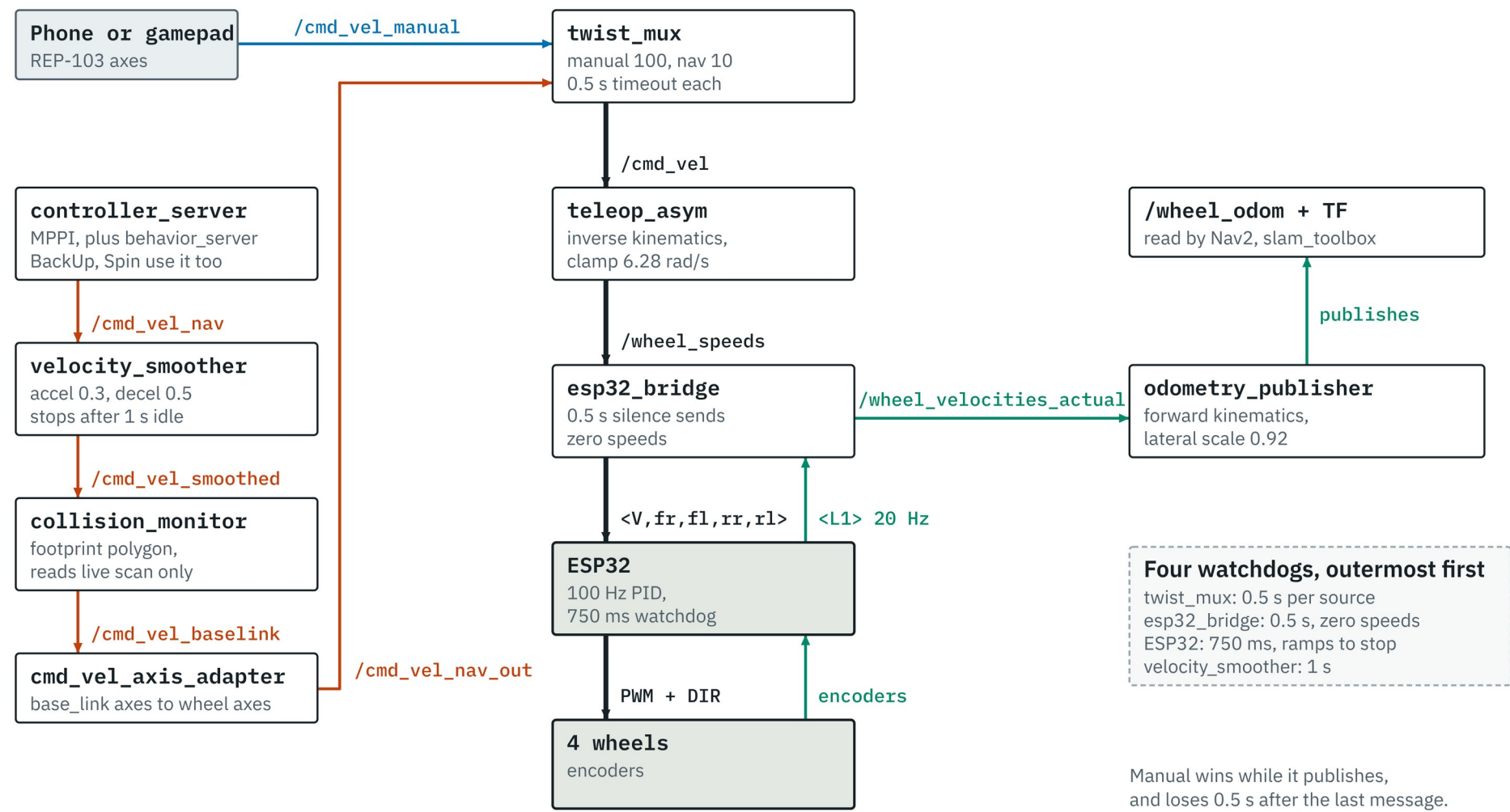
Started with Nav2: planning and control

Node	What it does	Why it is there
bt_navigator	Runs Nav2's stock behaviour tree: plan, follow, and on failure clear the costmaps, spin, back up, retry.	Standard Nav2, unmodified.
planner_server	NavFn with A* on the global costmap. Plans the whole route and may cross unknown cells.	The map grows while the robot drives, so the planner has to accept unknown space.
controller_server	MPPI on a 3 x 3 m rolling local costmap. Turns the path into velocity commands.	Its Omni motion model samples sideways velocity directly, which a mecanum base needs.
behavior_server	Recovery moves: Spin, BackUp, Wait.	On this robot BackUp is a 0.30 m strafe to the left, because Nav2 assumes +X is forward and here +X is right.
smoother_server, waypoint_follower, lifecycle_manager	Path smoothing, multi-goal runs, and bringing the Nav2 servers up in order.	Part of the standard set; not used much yet.

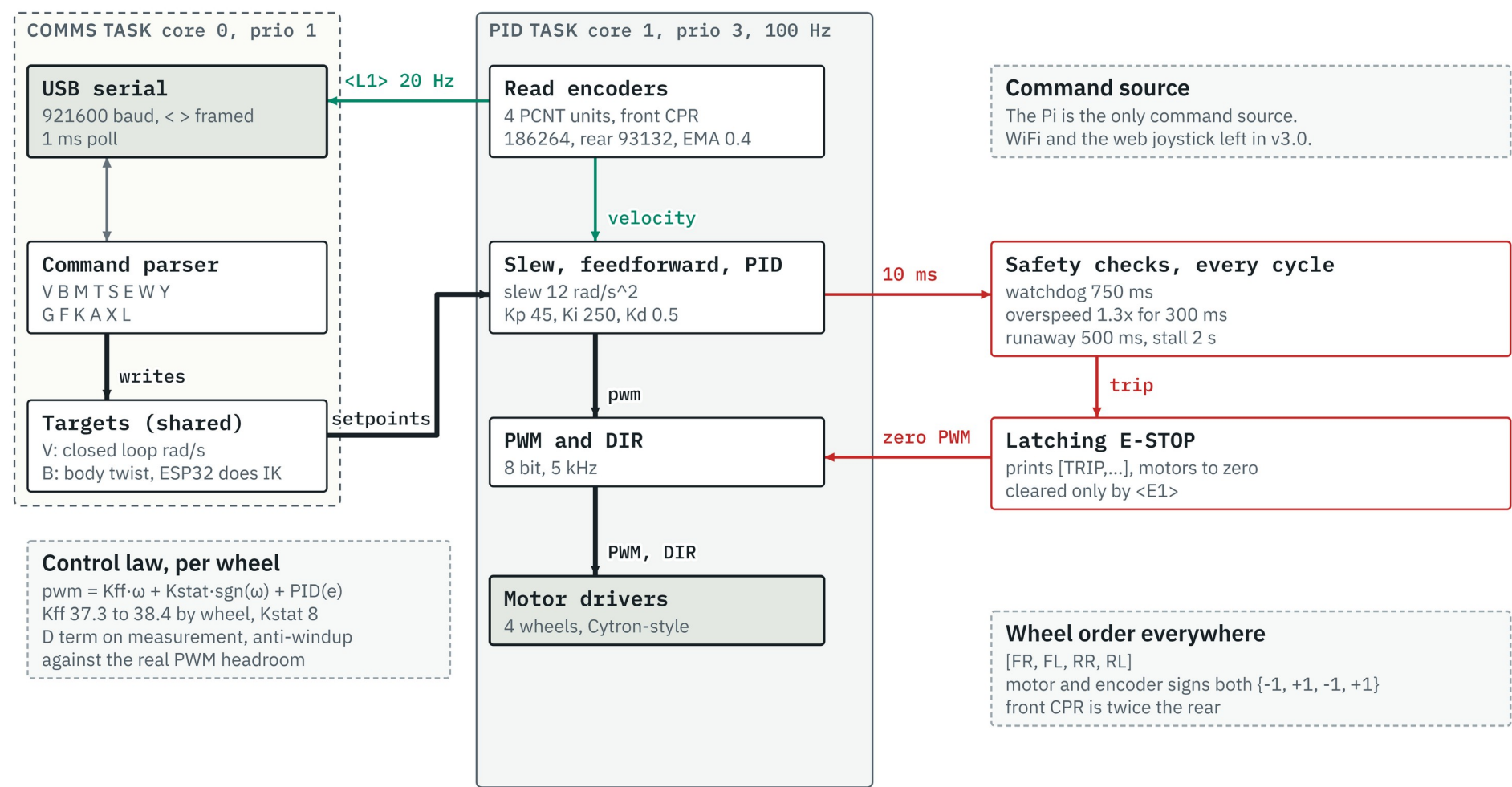
Started with Nav2: the safety chain and two adapters

Node	What it does	Why it is there
velocity_smoother	Caps speed at 0.12 m/s and 0.30 rad/s, limits acceleration to 0.3 m/s ² up and 0.5 m/s ² down, stops after 1 s without input.	Smooth, bounded commands whatever the controller asks for.
collision_monitor	Projects the robot's footprint 1.2 s ahead along the command and slows or stops if laser points fall inside.	A last check before the wheels that uses the live scan, not the map.
cmd_vel_axis_adapter written for this robot	Rotates the command from the map's axes (+X right, +Y forward) to the wheels' axes (+X forward, +Y left).	Everything upstream uses one axis convention and the wheel kinematics another.
goal_pose_adapter written for this robot	Turns a tap on the dashboard map into a Nav2 goal.	Lets the phone set goals.

Drive commands: two sources, one path to the wheels



Inside the ESP32 drive firmware



Odometry: the only source of pose

Inverse kinematics (teleop_asym and the ESP32)

$$\omega_{FR} = (v_x + v_y + K_{out} \cdot \omega_z) / r$$

$$\omega_{FL} = (v_x - v_y - K_{in} \cdot \omega_z) / r$$

$$\omega_{RR} = (v_x - v_y + K_{in} \cdot \omega_z) / r$$

$$\omega_{RL} = (v_x + v_y - K_{out} \cdot \omega_z) / r$$

Forward kinematics (odometry_publisher)

$$v_x = (r/4)(\omega_{FR} + \omega_{FL} + \omega_{RR} + \omega_{RL})$$

$$v_y = (r/4)(\omega_{FR} - \omega_{FL} - \omega_{RR} + \omega_{RL}) \times 0.92$$

$$\omega_z = (r/4)(\omega_{FR}/K_{out} - \omega_{FL}/K_{in} + \omega_{RR}/K_{in} - \omega_{RL}/K_{out})$$

$K_{out} = l_1 + d = 0.561$ m, $K_{in} = l_2 + d = 0.491$ m, $r = 0.0762$ m. Axes: x forward, y left.

0.229 m

closure error after a 21.85 m drive, wheel odometry alone
(3 Sep)

0.92

lateral scale factor: the rollers scrub when strafing. It was 0.80
on a different floor.

No IMU and no filter. Heading drift and strafe error are
not corrected by anything at present, and the forward
equations are the exact inverse of the ones the ESP32
drives with.

From LiDAR to map



SLAM as it runs today, and why scan matching is off

Setting	Value
Package	slam_toolbox 2.8.5, online asynchronous mapping
Solver	Ceres, sparse normal Cholesky, Levenberg-Marquardt
Scan matching	Off
Loop closing	On, 2.0 m search distance
New graph node	Every 0.2 m or 0.2 rad of travel
Laser range used	Up to 5.0 m
Map resolution	0.05 m per cell
Scan input	/scan_reliable from scan_relay

Same 21.85 m drive, error at the end

0.229 m wheel odometry alone

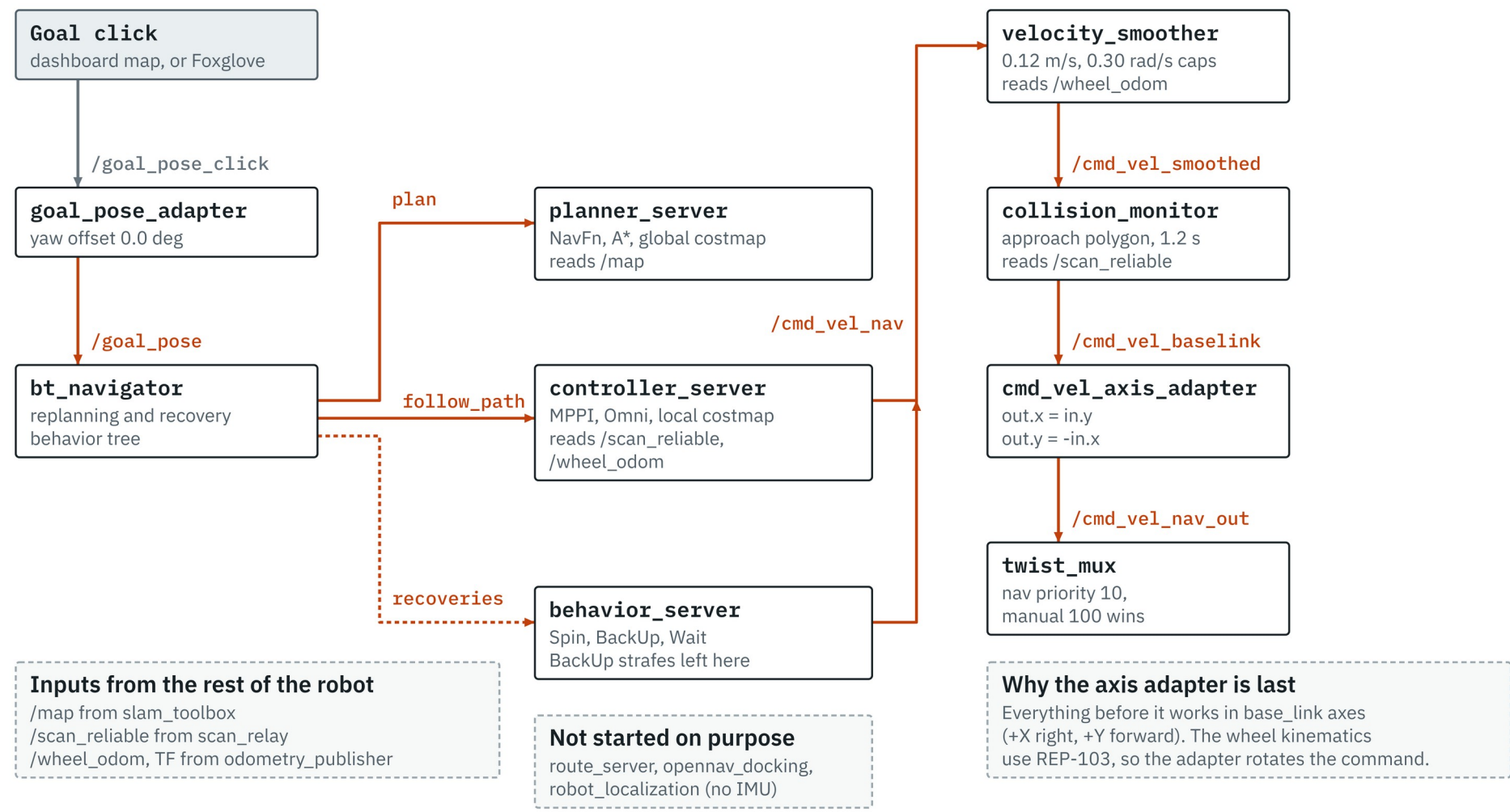
0.706 m odometry plus scan matcher

The scan matcher made pose three times worse, so it was switched off. That picks the better of two measured estimators.

The likely reason is on the input side: with the robot standing still, 75 to 78 % of rays flip between valid and invalid from one scan to the next.

Consequence: map to odom only moves when slam_toolbox accepts a new scan, about every 0.18 m. Mapping returns to the start mark within 6 to 30 mm, but 77 to 85 % of cells stay unknown, against a 50 % target (15 Sep).

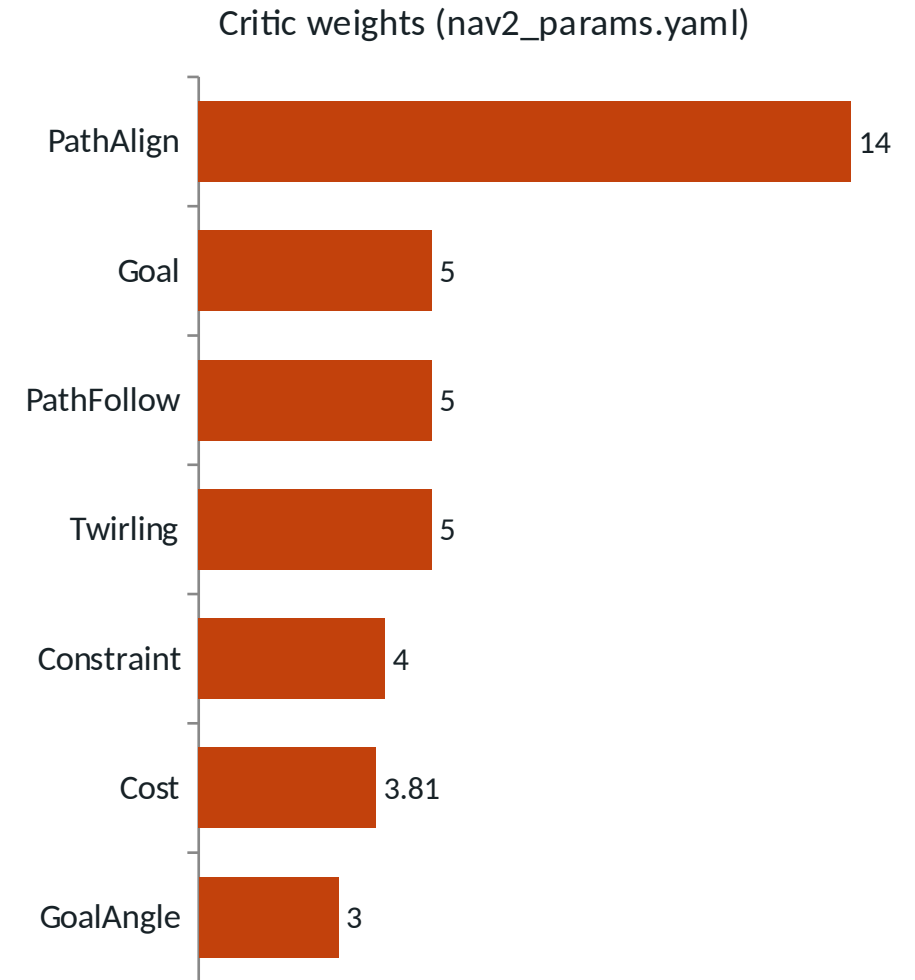
Nav2 as configured: from a tap to the wheels



MPPI: how the controller picks a command

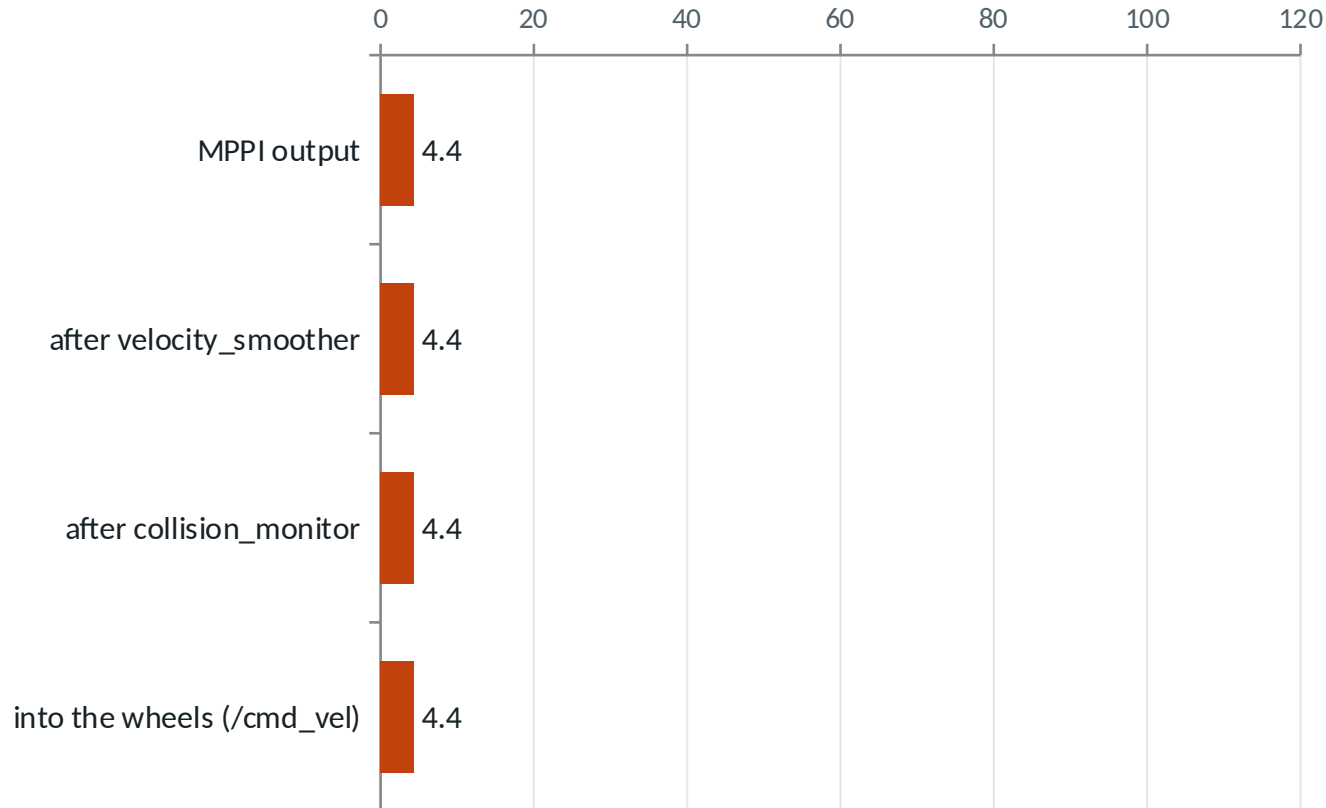
- 1 Start from last cycle's best control sequence: 40 steps of 0.05 s, so 2 s ahead.
- 2 Add random noise to make 300 candidates: $\sigma = 0.06$ m/s in x and y, 0.15 rad/s in yaw.
- 3 Roll each candidate forward with the Omni motion model, clipped to 0.12 m/s and 0.30 rad/s.
- 4 Score each with seven critics. A candidate that hits the costmap scores 1,000,000.
- 5 Average the candidates with weights $\exp(-(S - S_{\min})/\lambda)$, $\lambda = 0.3$. Send the first command, shift the sequence, repeat.

Requested at 20 Hz; measured 5 to 14 Hz on the Pi.



MPPI: where the speed goes missing (1 Oct test)

Speed requested at each stage, mm/s (axis ends at the 120 mm/s limit)



Mean over the first 40 s of the goal, from a bag of every stage of the chain.

The test

Goal 0.6 m straight ahead. Aborted after 194 s and 16 recoveries; the robot crept at about 1.5 mm/s. No contact.

What it rules out

Every stage after MPPI passes the same 4 mm/s on, so the smoother, collision monitor, axis adapter, mux and ESP32 are cleared.

What it points at

Averaging noise alone gives $0.06 / \sqrt{300} = 3.5$ mm/s, close to the 4.4 measured: the optimiser barely prefers any direction. The 0.3 m/s^2 acceleration limit is a suspect, not a proven cause.

Next

Same goal with Nav2's stock limits (3.0 m/s^2), already set on the live node.

Where things stand

Working, with measurements

- Wheel-speed loop on the ESP32 at 100 Hz, with latching safety trips
- Manual drive from the phone, with priority over Nav2 and four stacked watchdogs
- Wheel odometry: 0.229 m error over a 21.85 m drive
- Mapping: returns to the start mark within 6 to 30 mm
- Global planner: finds paths from the start mark (1 Oct)

Open

- No autonomous goal reached yet: MPPI asks for about 4 mm/s
- Map coverage: 77 to 85 % of cells unknown, against a 50 % target
- No IMU, so heading drift is not corrected
- A 90 degree blind arc behind the mast
- No hardware E-stop

Next steps, and where I would value your view

Next steps

1. Re-run the 0.6 m goal with stock MPPI acceleration limits.
2. If it drives, measure real acceleration on a straight run and set the limits from that.
3. If it still creeps, check temperature, sampling spread, critic weights and the real loop rate.
4. Then a ladder of goals: 1 m forward, 1 m sideways, a 90 degree turn, a reverse.
5. Rear sensing for the blind arc, a rear LiDAR or a ring of ToF sensors.

For an electrical review

E-stop. A latching normally-closed button in series with the SSR-50DD trigger. Is that the right place to break the circuit?

ESP32 supply. Pi USB today. Is VIN from the 5 V buck enough, or should the logic side be isolated?

Encoder edges. About 38.5 kHz per channel on the front encoders at the 5.2 rad/s limit, through a BSS138 shifter. Is the rise time comfortable?

Grounding. One bus shared by the 24 V drivers and the 3.3 V logic.

Appendix: every node and topic on one sheet

